

Architecting a Zero-DBU, CDC-Native Reverse ETL Pipeline

How we bypassed Databricks compute to stream Change Data Feed (CDF) events to downstream operational systems at scale.

TARGET_AUDIENCE: [Data Eng, Data Azch, Tech Lead] // CLEARANCE: Open

Gemini Notebook



TL;DR Execution Summary

Achieved Zero-DBU extraction bypassing Databricks compute via OSS Spark direct-to-storage reads.

Built native Debezium event streaming directly from Change Data Feed (CDF).

Delivered Multi-target syncs (Postgres + RisingWave) from a single Kafka topic, orchestrated by Dagster.

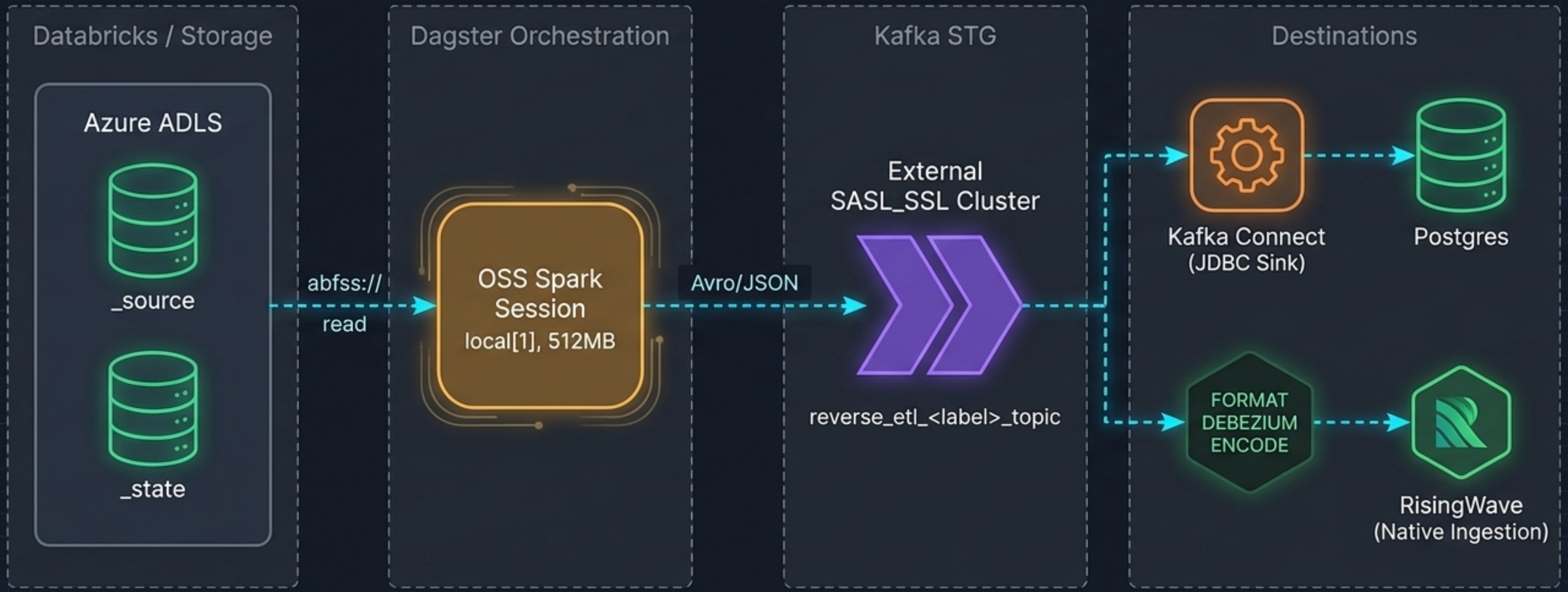
Gemini Notebook

Evaluation Criteria	drt (v1.0.0)	dlt	Custom Dagster Component
Destination Support	✗ Direct to DB only (No Kafka)	✗ Rejects Kafka destinations	✓ Writes to any Kafka consumer
Extraction Method	✗ Mirror mode (High DBU cost)	✗ Lacks CDF source	✓ Delta Change Data Feed (Pure delta)
Data Semantics	✗ Tracking tables (Loses explicit deletes)	✗ Row-level upserts	✓ Native Debezium CDC envelopes (c/u/d)

! Custom build required for CDC-native extraction, Kafka delivery, and zero-DBU scalability.

Gemini Notebook

End-to-end architecture from Delta storage to downstream consumers



Gemini Notebook

The Paid Path

table_changes()

Serverless SQL Warehouse ($$)

The Zero-Cost Path

PySpark 4.0.4 + Delta-Spark

Azure ADLS via ADLS_* credentials (Free)



[!] CONSTRAINT: The __state table must remain plain Delta (not Universal Format) because OSS Spark lacks the IcebergConverter class.

Gemini Notebook



CDF updates produce discrete rows. The pipeline accumulates lists per change type and groups positionally. This safely handles duplicate-keyed source rows mapping to the same commit.

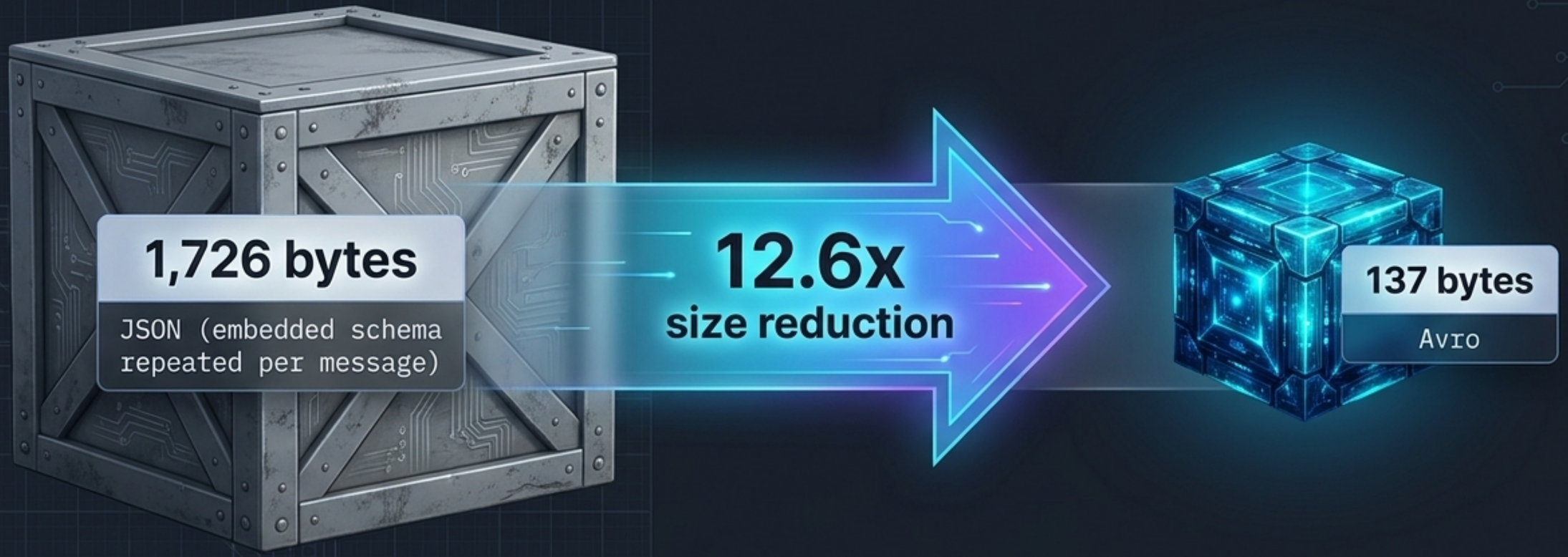
Gemini Notebook



All synced tables require rid BIGINT GENERATED ALWAYS AS IDENTITY.
This serves as the immutable message key.

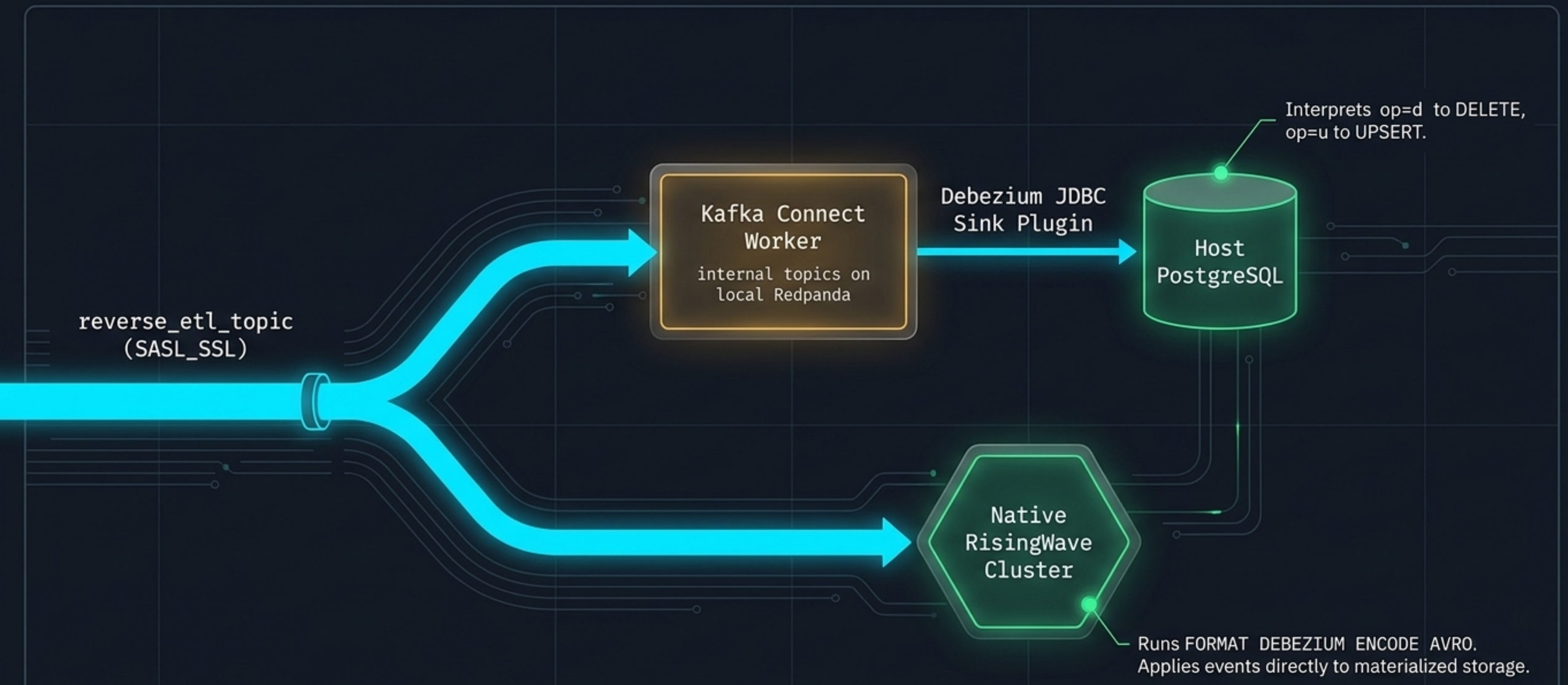
Gemini Notebook

Avro encoding shrinks message payloads by over 90 percent



Requirement:	Constraint:
Shared staging Apicurio registry.	RisingWave requires temporal workaround (timestamp ingested as VARCHAR, generated as TIMESTAMPTZ).

Gemini Notebook



One Topic, Multiple Readers. Each consumer tracks its own offsets independently.

Gemini Notebook

1

Databricks

`ALTER TABLE... ADD COLUMN`

2

Dagster Asset

Reads live Delta schema

3

Dagster Asset

Registers new schema version to Apicurio Registry

4

Dagster Asset

Executes `ALTER TABLE` on RisingWave

5

Dagster Asset

Produces Kafka events

6

Postgres Sink

Auto-adds column via schema.evolution=basic

Crucial Constraint: `RisingWave` must be altered before the first event carrying the new column is produced, otherwise ingested rows default to `NULL`. Evolution is additive only.

Gemini Notebook



The ReverseEtlSyncConfig object dynamically generates assets. A single label ensures zero name collisions, allowing isolated parallel syncs.

Gemini Notebook

Setup

Idempotent `IF NOT EXISTS` creation of tables and topics.

Sync

Manual or notebook trigger sharing a single Delta watermark.

One trigger per change.

Check

The `sink_healthy` scheduled asset check.
Prevents silent lag by polling the Connect REST API every 5 minutes.

Reset

Destructive teardown.

Safely isolated via strict `reset_name_prefixes` allowlists.

Gemini Notebook

Architectural heuristics: When to deploy which pipeline

What is your scale and requirement?

Deploy `drt` when:

- Handling small/medium reference tables.
- Syncing transformed SQL queries.
- Deletes are not required.
- Targeting a single database destination.
- Simplicity is prioritized.

Deploy Custom Pipeline when:

- Processing billions of rows with millions of daily changes.
- CDC streaming is required (zero-DBU reads).
- Multiple consumers need decoupled access via Kafka.
- Strict schema evolution tracking is mandatory.
- Replayable change history is needed.

Gemini Notebook